Technique Report of AQPHub

PEI MU, University of Edinburgh, United Kingdom
BITA ASOODEH, University of Edinburgh, United Kingdom
ANDERSON CHAVES CARNIEL, Huawei Technologies Research & Development (UK) Limited, United Kingdom
ANTONIO BARBALACE, University of Edinburgh, United Kingdom
AMIR SHAIKHHA, University of Edinburgh, United Kingdom

A clear and well-documented LaTeX document is presented as an article formatted for publication by ACM in a conference proceedings or journal publication. Based on the "acmart" document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.

ACM Reference Format:
Pei Mu, Bita Asoodeh, Anderson Chaves Carniel, Antonio Barbalace, and Amir Shaikhha. 2026. Technique Report of AQPHub. 1, 1 (March 2026), 3 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Experimental Setup

In this section, we describe the general setup of the evaluation, including the prototype, testbed, workloads, and the metrics we measure and evaluate.

DBMS Engines. We report performance with DuckDB (v1.3.2), PostgreSQL (v12.3), Umbra (v26.02), MariaDB (v12.2.2), and OpenGauss (v6.0.1).

Hardware. We conducted all experiments on a `x86/64` Dell server with a single Intel(R) Xeon(R) E-2236 CPU at 3.40GHz, 64 GB DDR4 RAM at 2666 MHz, 1 × 931.5 GB SATA HDD (6GB/s).

Software. We compiled the source code with gcc-11.4.0 on Ubuntu 22.04.5 LTS.

Measurement tools and method. For the end-to-end time, we use the hyperfine tool [2] with 5 times warmup, and we consider the average of 10 runs. For the performance breakdown, we use the timer function with the high-resolution clock in C++ [1]. We run `ANALYZE` before the measurement. The remaining settings are the same as [3].

2 Evaluation Analysis

This Section provides a performance analysis of AQPHub across five engines, including DuckDB, PostgreSQL, Umbra, MariaDB, and OpenGauss. We evaluate with two SOTA plan-based AQP algorithms, FK-Center from the QuerySplit [4] and the node-based split method from the AQP-DuckDB [3].

Authors' Contact Information: Pei Mu, University of Edinburgh, , United Kingdom, pei.mu@ed.ac.uk; Bita Asoodeh, University of Edinburgh, , United Kingdom, bita.asoodeh@ed.ac.uk; Anderson Chaves Carniel, Huawei Technologies Research & Development (UK) Limited, , United Kingdom, anderson.chaves.carniel@huawei.com; Antonio Barbalace, University of Edinburgh, , United Kingdom, antonio.barbalace@ed.ac.uk; Amir Shaikhha, University of Edinburgh, , United Kingdom, amir.shaikhha@ed.ac.uk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/3-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

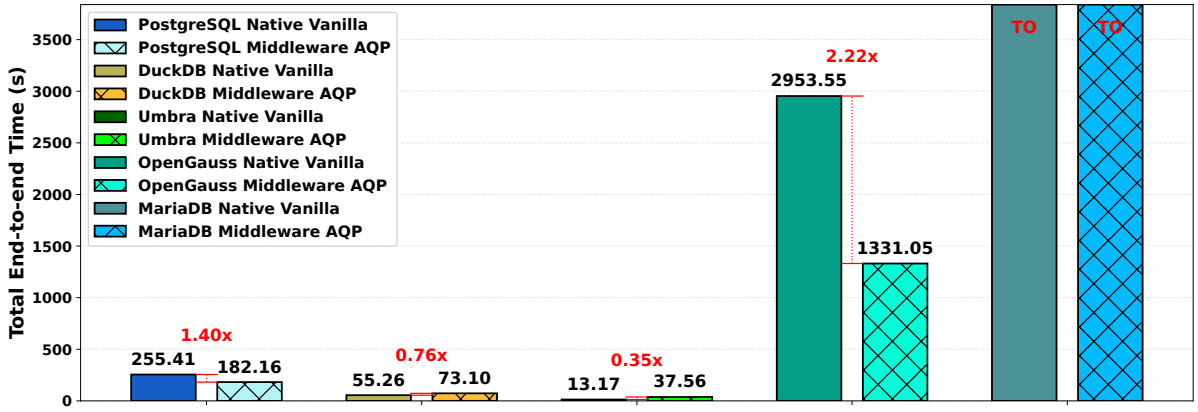

Fig. 1. Performance comparison of the FK-Center method between the vanilla DBMS and AQPHub across five engines on the JOB benchmark.

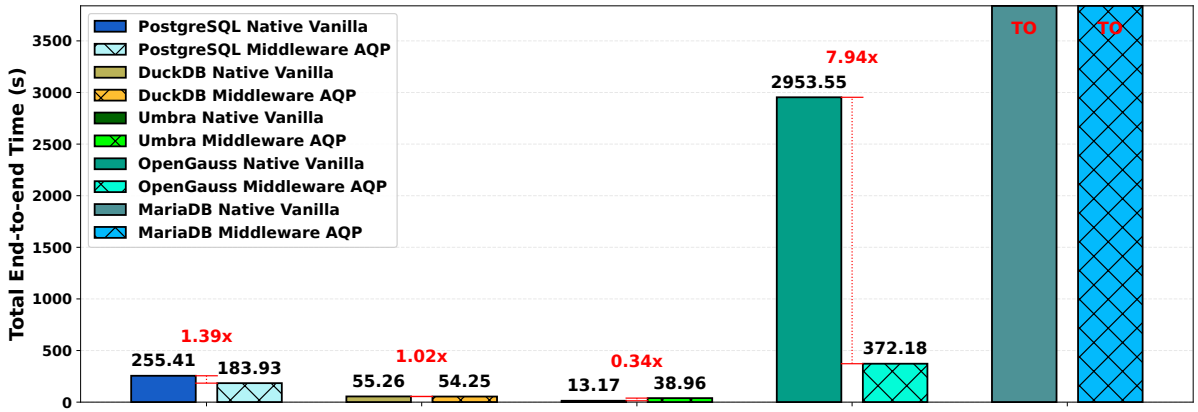

Fig. 2. Performance comparison of the node-based method between the vanilla DBMS and AQPHub across five engines on the JOB benchmark.

Figure 1 shows that for PostgreSQL and OpenGauss, AQPHub with FK-based split strategy achieves 1.40× improvement and 2.22× separately, whereas DuckDB and Umbra get a slowdown compared to the vanilla DBMS. MariaDB gets a timeout with both vanilla DBMS and AQPHub.

AQPHub with another split strategy, node-based split, shows improvements for both PostgreSQL, DuckDB, and OpenGauss. It is worth noting that the performance improvement in OpenGauss with the node-based split is better than that with the FK-Center split. However, it still gets a slowdown with Umbra. This is because the Umbra's optimizer and compilation improvement are already quite good. It is hard to explore in a further detailed way because the Umbra is closed-sourced and we cannot break down its performance.

References

[1] 2024. *C++ timer function std::chrono::high_resolution_clock*. Retrieved 18-12-2024 from https://en.cppreference.com/w/cpp/chrono/high_resolution_clock

[2] 2024. *Hyperfine github repo*. Retrieved 18-12-2024 from https://github.com/sharkdp/hyperfine

[3] Pei Mu, Anderson Chaves Carniel, Antonio Barbalace, and Amir Shaikhha. 2025. [Experiment, Analysis, and Benchmark] Systematic Evaluation of Plan-based Adaptive Query Processing. *arXiv preprint arXiv:2511.16455* (2025).

[4] Junyi Zhao, Huanchen Zhang, and Yihan Gao. 2023. Efficient Query Re-optimization with Judicious Subquery Selections. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–26.